MODULE 14 · DOMAIN 2 - APPLICATIONS AND INTEGRATION

JSON, Async and Concurrency

Serialising data safely, many Claude calls in flight with async code, bounded concurrency, timeouts, background jobs, and telling your bugs from the model's.

DOMAIN 2 of 8 EXAM WEIGHT 33.1% MODULE 14 of 36

admin@sthithapragna.com

A good cook never stands watching one pan. Async lets one cook tend many pans. A semaphore is the number of burners.

Where JSON appears in a Claude app

- **Request bodies:** the SDK serialises them. You pass Python or TypeScript objects.
- **Tool input:** arrives already parsed, as an object in the tool_use block.
- **Tool results:** content is a string or content blocks, so serialise your data yourself.
- **Structured outputs** and **streamed partial JSON:** modules 7 and 12.
- **Never build JSON by joining strings.** One quote in a name breaks it.

> THE TAKEAWAY
>
> Use the JSON library in both directions. Validate what comes in, serialise what goes out.

Returning a tool result safely

```
1  tenancy = get_tenancy(block.input["tenancy_id"])    # input arrives parsed
2  tool_results.append({
3      "type": "tool_result",
4      "tool_use_id": block.id,
5      "content": json.dumps({
6          "tenancy_id": tenancy.id,
7          "rent_pence": tenancy.rent_pence,           # money as integer pence
8          "next_due": tenancy.next_due.isoformat(),   # dates as ISO 8601
9      }),
10 })
```

5 The library escapes quotes, newlines and unicode for you.

7-8 Unambiguous types: whole pence, standard dates.

THE TIEBREAKER

Choose formats that cannot be misread: integers for money, ISO 8601 for dates, explicit nulls for unknowns.

Async clients

- **AsyncAnthropic** in Python: await client.messages.create(...). Streaming has the same interface.
- **Claude calls are I/O-bound:** mostly waiting on the network, so one process can keep many in flight.
- **Never call the sync client inside async code.** It blocks the event loop and stalls everything.
- **gather** starts many calls together; return_exceptions keeps one failure from losing the rest.
- **Optional aiohttp backend** for heavier async workloads.

> ON THE EXAM
>
> Async adds concurrency, not speed per call. Each call takes as long as it did.

Many calls, bounded

```python
client = AsyncAnthropic(max_retries=3, timeout=60.0)
limit = asyncio.Semaphore(10)   # sized to your rate limits

async def triage(ticket):
    async with limit:           # at most 10 in flight
        return await client.messages.create(
            model=TRIAGE_MODEL, max_tokens=1024,
            system=TRIAGE_PROMPT, messages=[as_message(ticket)])

results = await asyncio.gather(*(triage(t) for t in batch), return_exceptions=True)
```

2, 5 The semaphore is the burner count: never more than 10 calls at once.

10 Failures come back as values, to be retried or requeued.

THE TAKEAWAY

Bounded concurrency turns a burst into a steady flow, which is what module 13's limits want.

Back-pressure: don't let work pile up

- **Unbounded:** gather over fifty thousand items creates fifty thousand tasks at once.
- **Bounded queue and worker pool:** producers wait when the queue is full.
- **Process in chunks,** saving each result as it completes.
- **Partial failure is normal:** requeue failed items by ID, as in module 10.
- **Large and nobody waiting?** That is still a Batch job, not an async one.

> THE TIEBREAKER
>
> Back-pressure means slow producers down when consumers fall behind, instead of buffering forever.

Timeouts and cancellation

FACT	WHAT TO DO
SDK default timeout: 10 minutes	Far too long for a person waiting. Set your own
Set per client or per request	with_options(timeout=…) for one call
Timeouts are retried twice by default	Worst case is roughly three timeouts back to back
Long outputs without streaming	Idle connections may drop: stream instead
A person gives up	Cancel the task or close the stream

ON THE EXAM

For user-facing calls, set an overall deadline in your own code, and show a graceful fallback when it passes.

Request thread or background job?

IN THE REQUEST

Someone is waiting

- Short, interactive calls
- Streamed to the user
- A tight overall deadline
- Fails fast with a fallback

BACKGROUND JOB

Accept now, finish later

- Long documents, multi-step or bulk work
- Queued, keyed by an idempotency ID
- Retries without anyone waiting
- Status or a notification when done

RUNNING EXAMPLE

Copperlet's document reader: upload returns at once; a job keyed by document ID reads it; the agent is notified.

Your bug, or the model's?

- **Log the raw request and raw response,** with the request ID, before any parsing.
- **Replay the recorded response** through your code, with no API call. Still fails? It is your code.
- **Check the stop reason** and schema validation before blaming content.
- **Keep recorded fixtures** as deterministic tests for the integration layer.
- **Only then** look at prompts, models and examples.

> THE TIEBREAKER
>
> Separate the integration layer from model output first. Rule two: evidence before change.

> A Python web service with async endpoints calls Claude using the synchronous client. Under moderate load, every endpoint slows to a crawl, including ones that never call Claude.
>
> CPU usage is low.

What is the most likely fix?

A Add more server processes, without changing the code

B Use the async client and await its calls, so the event loop is not blocked

C Raise the request timeout so that slow calls can finish

D Move every Claude call to the Message Batches API

◆ PAUSE AND DECIDE

What is the most likely fix?

A Add more server processes, without changing the code

Each process still blocks the same way.

B Use the async client and await its calls, so the event loop is not blocked

Waiting no longer freezes everything else.

C Raise the request timeout so that slow calls can finish

Longer blocking, not less.

D Move every Claude call to the Message Batches API

Users are waiting on these calls.

THE SIGNAL

Async endpoints, a sync client, low CPU, everything slow: a blocked event loop.

> A script summarises 20,000 documents by passing all of them to a single gather call. Within seconds it uses several gigabytes of memory, and most calls fail with 429 errors.
>
> The documents must all be summarised by tomorrow.

What is the best change?

A. Keep the single gather, and add more retries to each call

B. Process the documents strictly one at a time

C. Bound concurrency with a semaphore or worker pool, save results as they finish, and requeue failures

D. Request a higher rate-limit tier, and keep the single gather

◆ PAUSE AND DECIDE

What is the best change?

A Keep the single gather, and add more retries to each call

More retries feed the same storm.

B Process the documents strictly one at a time

Safe, but may miss tomorrow.

C Bound concurrency with a semaphore or worker pool, save results as they finish, and requeue failures

Steady flow, bounded memory, nothing lost.

D Request a higher rate-limit tier, and keep the single gather

The burst and memory problems remain.

THE SIGNAL

Memory and 429s at once from one unbounded gather: bound it. A deadline of tomorrow would also suit Batch.

A booking tool builds its tool result by inserting values into a JSON-shaped string. When a customer's name contains a quotation mark, Claude receives broken JSON and replies with confusion.

The team wants a permanent fix.

What should the developer do?

A Build the result as a normal object and serialise it with the JSON library

B Tell Claude in the system prompt to tolerate malformed JSON in tool results

C Strip quotation marks from customer names before inserting them

D Switch the tool result to a hand-built XML string instead

◆ PAUSE AND DECIDE

What should the developer do?

A Build the result as a normal object and serialise it with the JSON library

Escaping handled correctly, every time.

B Tell Claude in the system prompt to tolerate malformed JSON in tool results

Asks the model to absorb your bug.

C Strip quotation marks from customer names before inserting them

Corrupts real data, and misses other characters.

D Switch the tool result to a hand-built XML string instead

The same mistake in another format.

THE SIGNAL

Hand-built JSON is the bug. Serialise with a library, in every direction.

> A user-facing assistant uses the SDK's default timeout and retry settings. When the service is slow, some users wait several minutes before seeing any error.
>
> The product team wants users to get a response or a clear fallback within seconds.

Which two changes should the developer make? Choose two.

A. Set an explicit timeout suited to this feature, per client or per request

B. Stream responses, and enforce an overall deadline in code with a graceful fallback

C. Keep the default ten-minute timeout, which suits every workload

D. Disable retries across every service in the company

E. Raise max_tokens so responses complete in a single call

◆ PAUSE AND DECIDE

Which two changes should the developer make? Choose two.

A Set an explicit timeout suited to this feature, per client or per request

Defaults suit long jobs, not people waiting.

B Stream responses, and enforce an overall deadline in code with a graceful fallback

First words fast; a bounded worst case.

C Keep the default ten-minute timeout, which suits every workload

That default is the cause.

D Disable retries across every service in the company

Far too broad. Tune them for this feature.

E Raise max_tokens so responses complete in a single call

Longer outputs, longer waits.

CURRENCY CHECK

Defaults: 10-minute timeout, retried twice. For a person waiting, choose your own numbers.

> An extraction service sometimes throws a parsing error. The team believes Claude is returning bad output and plans to rewrite the prompt and switch models.
>
> Raw responses and request IDs are logged before parsing.

What should the team do first?

A. Switch to a different model, since the parsing errors began recently

B. Rewrite the prompt with stronger formatting instructions

C. Replay a logged failing response through the parsing code, to see whether the fault is in the integration or the output

D. Add retries around the parser until it succeeds

◆ PAUSE AND DECIDE

What should the team do first?

A Switch to a different model, since the parsing errors began recently

A change before any evidence.

B Rewrite the prompt with stronger formatting instructions

Fixes the model's side, which may be fine.

C Replay a logged failing response through the parsing code, to see whether the fault is in the integration or the output

Isolates the layer, cheaply and exactly.

D Add retries around the parser until it succeeds

Hides the fault, whichever side it is on.

THE SIGNAL

"Raw responses are logged" is the signal: the evidence is already there. Replay it first.

> A document upload endpoint calls Claude synchronously to read 60-page contracts. The company's gateway cuts requests off after 30 seconds, so users see errors and upload again, creating duplicate records.
>
> Results are not needed instantly.

What is the best design?

A. Raise the gateway timeout to ten minutes for this endpoint

B. Accept the upload, queue a background job keyed by document ID, return at once, and notify when done

C. Reject contracts longer than ten pages

D. Call Claude twice in parallel and keep whichever finishes first

◆ PAUSE AND DECIDE

What is the best design?

A Raise the gateway timeout to ten minutes for this endpoint

Users still wait, and still retry.

B Accept the upload, queue a background job keyed by document ID, return at once, and notify when done

No timeout, no duplicates, no waiting.

C Reject contracts longer than ten pages

Breaks the feature to fit the plumbing.

D Call Claude twice in parallel and keep whichever finishes first

Double cost, same timeout.

THE SIGNAL

Long work, a gateway limit, and "not needed instantly": accept now, finish in a background job.

Module 14 in five lines

- **Serialise with a library;** tool input arrives parsed, tool results you serialise.
- **Async client, awaited;** never the sync client inside async code.
- **Bound concurrency** with a semaphore or queue; requeue failures.
- **Choose your timeouts:** defaults are 10 minutes, retried twice.
- **Long work goes to a background job;** replay raw responses to find whose bug it is.

> COMING UP
>
> Next: tests, version control, code review and refactoring.